

Lab 2B: Hooks and Commands

Duration: 35 min

After: Hooks & Custom Commands lecture (Day 2, Block 2)

Prerequisites: Lab 2A complete (code-review skill)

Objective

Pick the right event from the **27 hook events**, wire a **PostToolUse command hook** that runs on every edit, add a PreToolUse secret blocker, and build one **parameterized custom command** using **\$ARGUMENTS**.

Deliverable (toolkit chain 6 of 12)

Toolkit artifacts #3 and #4: a registered hook (`.claude/hooks/` + `settings.json`) and a parameterized command (`.claude/commands/review.md`).

START MARKER: *In business-dashboard, /clear done.*

Fell behind? Run:

Set up a code-review skill: create `.claude/skills/code-review/SKILL.md` with frontmatter (name, description with triggers "review/code review/find bugs/QA check", `disable-model-invocation: false`, `allowed-tools: Read, Grep, Glob`) and instructions producing RED/YELLOW/BLUE output. Also create a `CLAUDE.md` with basic project rules.

Exercise 0: Choose Your Event (5 min)

Hooks are **hard enforcement** - external handlers that run on lifecycle events, unlike skills which Claude merely follows. As of v2.1.141+ there are **27 events** in five groups:

Group	Events
Session	SessionStart, Setup, SessionEnd, InstructionsLoaded, ConfigChange, CwdChanged
Prompt/response	UserPromptSubmit, UserPromptExpansion, Stop, StopFailure, Notification, Elicitation, ElicitationResult
Tools	PreToolUse , PostToolUse , PostToolUseFailure, PostToolBatch, PermissionRequest, PermissionDenied
Agents & tasks	SubagentStart, SubagentStop, TeammateIdle, TaskCreated, TaskCompleted
Workspace/context	FileChanged, WorktreeCreate, WorktreeRemove, PreCompact, PostCompact

And **4 handler types**: command (shell), HTTP, MCP tool, prompt. Today you build two **command** handlers.

Warm-up - answer before building (check with your neighbor):

1. Auto-format a file after Claude edits it -> which event? (**PostToolUse**)
2. Block a write containing a secret BEFORE it lands -> (**PreToolUse**)
3. Log every subagent completion -> (**SubagentStop**)

Exercise 1: PostToolUse Format Hook (10 min)

The required hook for your toolkit: format every file Claude edits.

Step 1:

Create a post-tool hook at `.claude/hooks/post-edit-format.sh` that:

- Reads JSON from stdin: `json=$(cat)`
- Parses the file path: `FILE=$(echo "$json" | jq -r '.tool_input.file_path // empty')`
- Exits 0 if no file path
- Only processes `.js/.ts/.jsx/.tsx` files
- Runs `"npx prettier --write"` on the file if `npx` is available
- Make it executable with `chmod +x`

Then add a `PostToolUse` hook to `.claude/settings.json` in the current project directory with this structure:

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Write|Edit",
        "hooks": [ { "type": "command", "command": ".claude/hooks/post-edit-format.sh" } ] }
    ]
  }
}
```

Merge with existing settings - don't overwrite permissions.

Matchers are exact and case-sensitive: Write, Edit, Bash - not lowercase. "type": "command" is one of the 4 handler types.

Step 2 - reload:

```
/clear
```

Step 3 - test:

Add a comment to the top of `server.js` saying `"// Business Dashboard API Server"` but format it badly with inconsistent spacing.

Expected output marker: after the edit, the hook fires and Prettier normalizes the file. Verify: `!head -3 server.js` shows clean formatting.

Exercise 2: PreToolUse Secret Blocker (8 min)

Create a bash script at `.claude/hooks/pre-write-check.sh` (`chmod +x`) that:

- Reads JSON from stdin
- Extracts `CONTENT=$(jq -r '.tool_input.content // empty')`
- Exits 0 if empty
- If the content matches `(API_KEY|SECRET|PASSWORD|TOKEN)\s*=\s*"[^"]+"` or `sk-[a-zA-Z0-9]{20,}`, outputs:
`{"hookSpecificOutput":{"hookEventName":"PreToolUse","permissionDecision":"deny","permissionDecisionReason":"Hardcoded secret detected"}}`
- Otherwise exits 0

Register it in `.claude/settings.json` as a `PreToolUse` hook with matcher `"Write"`, type `"command"`.

Reload with `/clear`, then test both paths:

```
Create a file called test-secret.js with this line: const API_KEY = "sk-1234567890abcdef"
```

Expected output marker: write **blocked** - "Hardcoded secret detected".

Create a file called `test-safe.js` with this line: `const apiKey = process.env.API_KEY`

Expected output marker: write allowed. Clean up: Delete `test-safe.js`.

How blocking works: stdout JSON `permissionDecision: "deny"` blocks; exit 0 with no output allows. Skills advise - hooks enforce.

Exercise 3: Parameterized Command - /review (8 min)

Commands are explicit workflows you trigger with a slash; `$ARGUMENTS` makes them parameterized.

Create a custom command at `.claude/commands/review.md` with:

- YAML frontmatter: `description "Run a full code review on specified files"`
- Body: Run a comprehensive code review on `$ARGUMENTS` using the `code-review` skill workflow. After the review, produce: total issues (critical/warning/info), top 3 fixes, and a one-line verdict: `APPROVE`, `NEEDS CHANGES`, or `BLOCK`.

If no files are specified in `$ARGUMENTS`, review all changed files (`git diff --name-only`).

Reload with `/clear`, then test with and without a parameter:

```
/review src/routes/tasks.js
```

Expected output marker: RED/YELLOW/BLOCK review of that one file + a verdict line.

```
/review
```

Expected output marker: Claude runs `git diff --name-only` and reviews changed files - the `$ARGUMENTS` fallback branch.

Exercise 4: Commit the Toolkit (4 min)

Stage and commit `.claude/hooks/`, `.claude/commands/`, `.claude/skills/`, and `.claude/settings.json` with message "feat: add format hook, secret blocker, and `/review` command to toolkit"

FINISH MARKER - Success Checklist

- ☐ You can name the 5 event groups and 4 handler types
- ☐ `PostToolUse` format hook fires on `Write|Edit` and prettifies files
- ☐ `PreToolUse` hook blocks a hardcoded secret and allows env-var code
- ☐ Hooks registered in `.claude/settings.json` with exact-case matchers
- ☐ `/review <file>` and bare `/review` both behave correctly
- ☐ Toolkit committed

If Stuck

ite not write); `/clear` to reload; check `/doctor` - July 2026 build flags slow/broken

install jq / winget install jqlang.jq) or ask Claude to rewrite the hook with	
ually: `!echo '{"tool_input":{"content":"hello"}}`	.claude/hooks/pre-write-check.sh` - expect exit 0
st be .claude/commands/review.md in the PROJECT; /clear to reload	
--save-dev prettier	

Debrief Questions

- 1. Skills vs hooks: which one can Claude choose to ignore, and why does that distinction matter for compliance?
- 2. Which of the 27 events would you wire first in your real repo, with which of the 4 handler types?
- 3. What does \$ARGUMENTS let a command do that a skill description can't?